Platform Engineer 포트폴리오

강희주

현대오토에버 Platform Engineer(컨테이너 플랫폼 구축 및 운영) 업무와 연결되는 경험

Kubernetes 기반 플랫폼 제어와 Stateful 환경 운영, 반복 검증 및 장애 분석을 중심으로 경험을 쌓았습니다.

업무 연결 포인트	관련 경험
① Kubernetes 플랫폼 운영	3-node Kubernetes 클러스터 운영 / StatefulSet · Service · PV/PVC · Longhorn CSI 기반 Stateful 환경 구성
② CRD · Operator 기반 플랫폼 제어	VolumeSwitch CRD 및 Go Operator 설계 · 구현 / Kubernetes 리소스 상태 기반 전환 자동화
③ 기능 테스트 · 검증 자동화	반복 테스트 자동화 / 서비스 가용성 · 데이터 일치 여부 · 리소스 상태 자동 검증
④ OSS 적용 · Troubleshooting	Longhorn CSI 적용 및 운영 / Kubernetes 리소스 상태와 생명주기 기반 오류 원인 분석 · 개선
⑤ Linux · Cloud 환경 경험	Rocky Linux 기반 Kubernetes 환경 운영 / Terraform 기반 AWS EKS 인프라 · 접근 권한 환경 구축

01 Kubernetes Stateful 워크로드 무중단 볼륨 축소 설계·구현

Kubernetes Stateful 환경에서 무중단 볼륨 전환을 구현하며,
제어 로직부터 클러스터 구성 · 자동화 · 검증까지 수행했습니다.

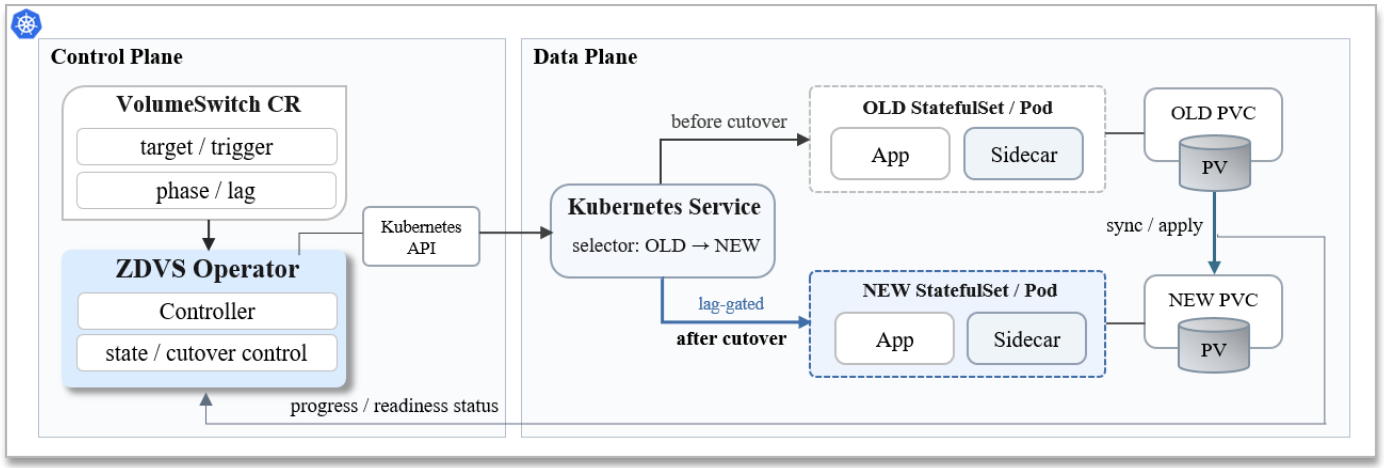
Kubernetes | Persistent Storage | Automation | Troubleshooting

문제 정의

Kubernetes는 볼륨 축소를 기본 지원하지 않아, 더 작은 볼륨으로 교체하려면 워크로드 재구성과 서비스 중단이 발생

수행 역할

아키텍처 설계	인프라 환경 구축	자동화 & 검증
볼륨 전환 CRD 및 Operator 설계 · 구현	StatefulSet · PV/PVC · Longhorn CSI · MySQL 환경 구성	4×4 매트릭스 반복 실행 자동화
동기화 잔여량 · 신규 Pod 준비 상태 기반 State Machine 설계	3노드 Kubernetes 클러스터 운영	전환 · 데이터 정합성 · 리소스 상태 자동 수집
Service selector 기반 기존 → 신규 볼륨 무중단 전환	대용량 검증을 위한 Longhorn 스토리지 확장	실행 상태와 결과를 분리한 검증 조건 구성



기존 재시작 기반 방식 **69–125s** 중단
→ 제안 방식 **0ms**

01 Kubernetes Stateful 워크로드 무중단 볼륨 축소 설계·구현

Kubernetes 리소스 상태를 기준으로 볼륨 전환 자동화

전환 조건을 Custom Resource에 정의하고, Go Operator가 동기화 상태와 신규 Pod의 준비 상태를 확인해 Service 전환을 수행하도록 구현했습니다.

전환 조건 관리 | VolumeSwitch CRD

전환 대상
기존·신규 StatefulSet / 전환할 Service

전환 조건
허용 가능한 동기화 차이 / 안정 상태 유지 시간 / 전환 실행 요청

진행 상태
현재 단계 / 남은 동기화량 / 상태 변경 시점

전환에 필요한 설정과 현재 진행 상태를 하나의 Kubernetes 리소스로 관리

상태 확인 및 전환 | Go Operator

Observe
남은 동기화량 / 신규 Pod 정상 여부 / 현재 Service 대상

↓

Decide
설정한 전환 조건 충족 여부 확인

↓

Act
진행 상태 갱신 / Service 대상을 NEW로 변경

Go · client-go · Reconcile Loop

제어와 동기화 역할 분리

Operator
언제 전환할지 판단 / 전환 단계 관리 / Kubernetes API 호출

Sidecar
OLD의 변경 감지 / NEW에 변경 내용 반영 / 동기화 상태 전달

Operator (Control Plane) / Sidecar (Data Plane)

Sidecar는 데이터 동기화,
Operator는 전환 판단과 실행 담당

CRD와 Go Operator로 상태 확인 → 전환 판단 → Service 변경까지 하나의 제어 흐름으로 구현했습니다.

01 Kubernetes Stateful 워크로드 무중단 볼륨 축소 설계·구현

Troubleshooting

서비스는 정상인데 검증 작업만 간헐적으로 실패하는 현상을 추적해, 이전 실행의 Pod가 잘못 선택되는 문제를 확인하고 개선했습니다.

01 증상

서비스는 정상 / 검증 작업은 실패

반복 실행 중 timeout / failed 간헐 발생
실제 Service 요청과 Probe Pod는 정상

→ 서비스 장애가 아닌 검증 절차 문제로 범위 축소

02 원인

같은 Job 이름을 반복 사용해 이전 실행의 Pod까지 조회

같은 Job 이름 반복 사용
↓
job-name으로 Pod 조회
↓
목록의 첫 Pod 선택
↓
삭제 중인 이전 Pod 선택 가능

현재 실행과 이전 실행을 구분하는 기준 부족

03 개선

고유 ID로 현재 실행의 Pod만 추적

Job 생성
↓
Job UID 저장
↓
Pod의 ownerReference와 UID 비교
↓
현재 Job의 Pod만 선택

이전 Custom Resource의 완전 삭제 확인 후 다음 실행
Pod 상태, 로그 기준으로 완료 여부 확인

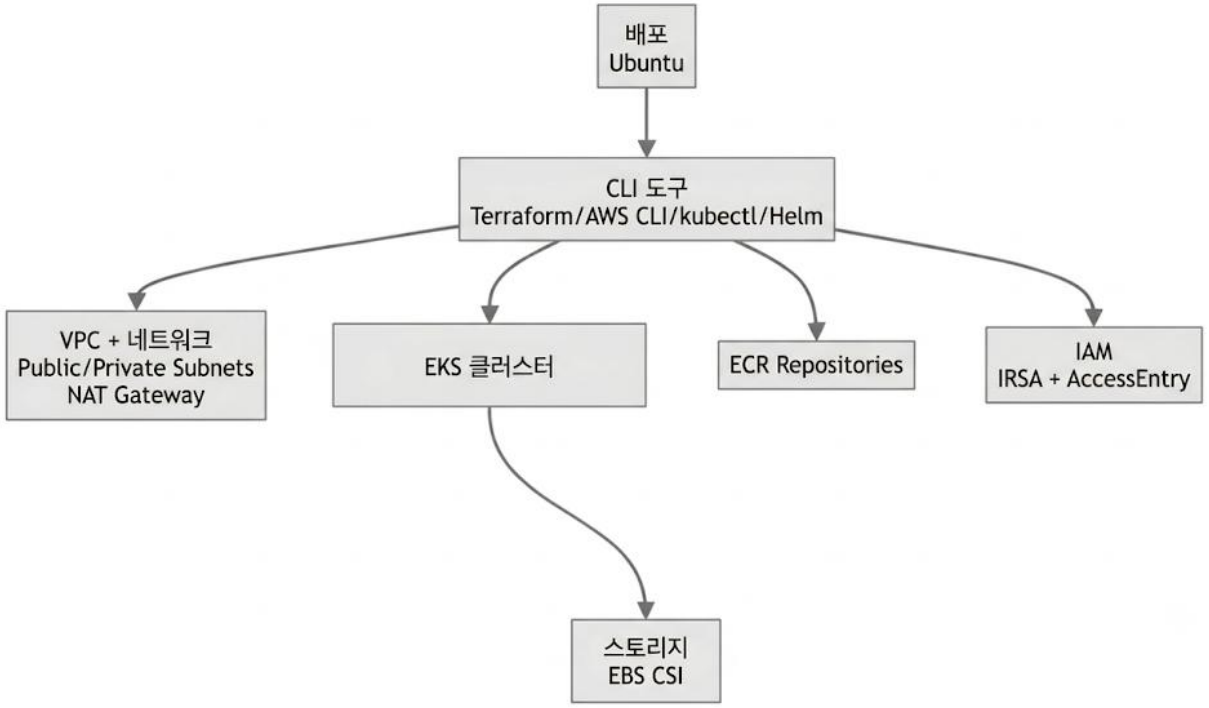
04 결과

동일 조건 재실행에서 timeout 오류 미재현

02 Terraform 기반 AWS EKS 인프라 구축 및 운영 자동화

AWS | Terraform | IAM/RBAC | Automation | Collaboration

여러 서비스가 공통 EKS 환경에 배포될 수 있도록 AWS 네트워크부터
Kubernetes · 스토리지 · 이미지 저장소 · 사용자 접근 체계까지 플랫폼 기반 인프라를 구성했습니다.



수행 역할

네트워크	VPC · 2AZ Public/Private Subnet · NAT 구성 ALB Ingress 생성 및 외부 접근 경로 검증
클러스터 & 스토리지	Amazon EKS 환경 구축 EBS CSI · StorageClass/PVC 구성 및 실제 Pod 쓰기 검증 ECR 이미지 저장소 구성
접근 권한	AWS 인증과 Kubernetes 권한 연계 사용자별 kubeconfig/context 구성 및 접근 범위 검증
자동화	Terraform 기반 AWS 인프라 구성 Shell 기반 초기화 · 검증 · 회수 절차 자동화

02 Terraform 기반 AWS EKS 인프라 구축 및 운영 자동화

인프라 구축 · 검증 자동화

수동 구축 절차를 Shell Script 기반 5단계로 자동화하고, 문제 발생 시 단계별 확인 · 재실행이 가능하도록 구성

01 도구 준비 → 02 환경 설정 → 03 인프라 구축 → 04 클러스터 초기 설정 → 05 배포 검증

도구 준비 — Terraform / AWS CLI / kubectl / Helm 설치 · 확인

환경 설정 — env 기반 환경값 분리

인프라 구축 — VPC / Subnet / NAT / EKS / ECR / EBS CSI

클러스터 초기 설정 — Namespace / Access Entry / RBAC

배포 검증 — Node / Pod / Service / Ingress / Storage

의존성을 고려한 단계적 리소스 회수

Kubernetes 리소스 → EKS Access Entry → Terraform 관리 인프라 → ECR → kubeconfig/env 순으로 정리

접근 권한 구성

AWS 인증 주체와 Kubernetes RBAC을 연결해 담당자별 Namespace 접근 범위를 구성

IAM User → EKS Access Entry → Kubernetes Group → RoleBinding → Namespace

5.2 context 목록 확인

```
$ kubectl config get-contexts
```

출력 예:

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	d1-dev	homepot-platform	D1-user	d1-dev
	d2d3-dev	homepot-platform	D2D3-user	d2d3-dev

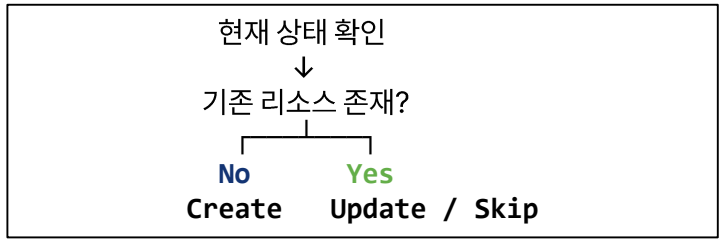
사용자별 kubeconfig/context 구성 확인

02 Terraform 기반 AWS EKS 인프라 구축 및 운영 자동화

재실행 가능한 운영 자동화

반복 실행 문제
기존 Access Entry가 존재하면 신규 생성 명령 실패

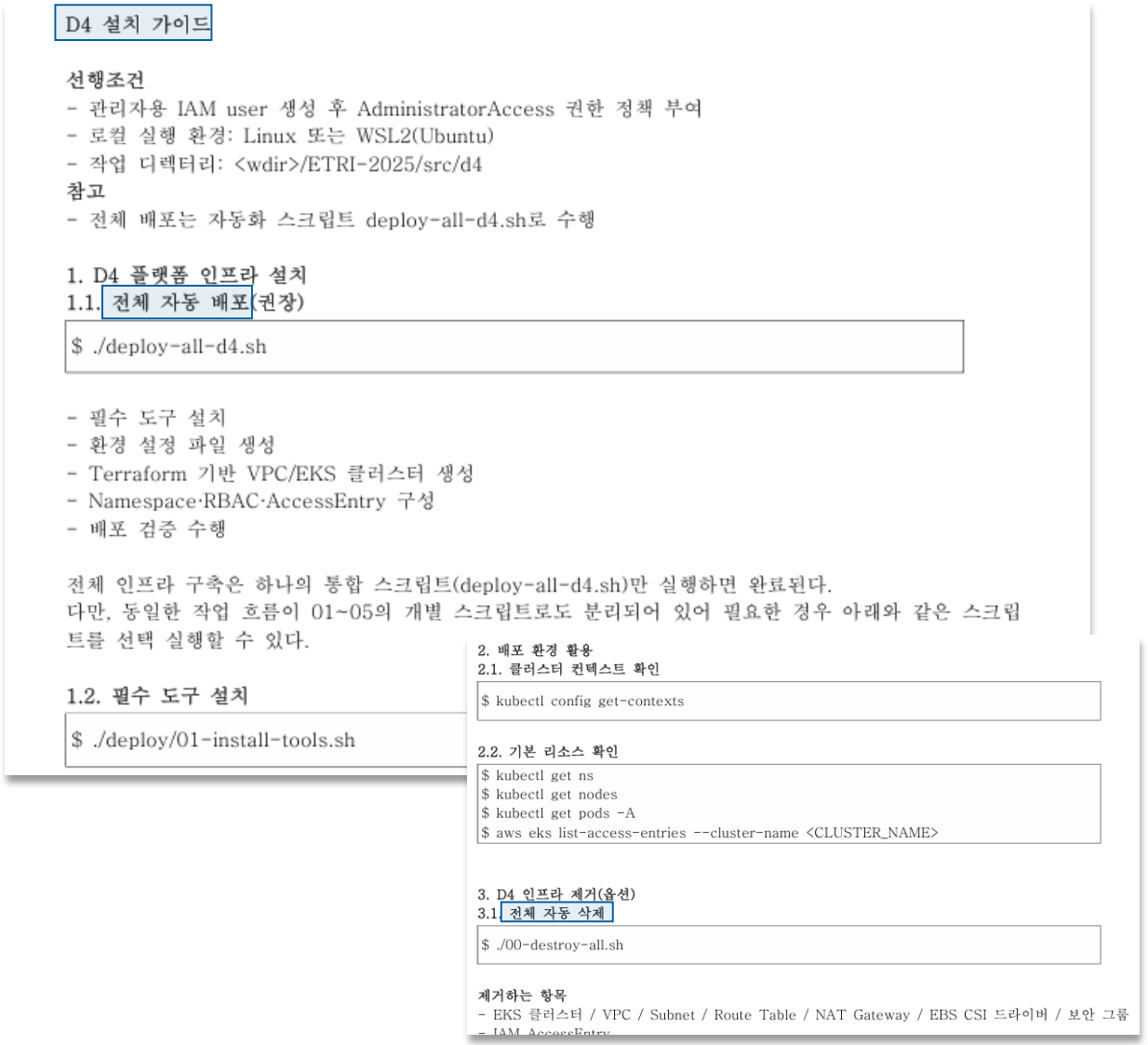
개선
기존 리소스 상태를 먼저 확인해 Create / Update / Skip으로 분기



결과
중복 생성과 리소스 충돌을 방지해 재실행 가능한 구축 절차 구성

문서화

팀원이 동일 환경을 재현할 수 있도록 설치/사용자 가이드 및 기능 설계서 작성 후 공유



D4 설치 가이드

선행조건
- 관리자용 IAM user 생성 후 AdministratorAccess 권한 정책 부여
- 로컬 실행 환경: Linux 또는 WSL2(Ubuntu)
- 작업 디렉터리: <wdir>/ETRI-2025/src/d4

참고
- 전체 배포는 자동화 스크립트 deploy-all-d4.sh로 수행

1. D4 플랫폼 인프라 설치
1.1. 전체 자동 배포(권장)

```
$ ./deploy-all-d4.sh
```

- 필수 도구 설치
- 환경 설정 파일 생성
- Terraform 기반 VPC/EKS 클러스터 생성
- Namespace·RBAC·AccessEntry 구성
- 배포 검증 수행

전체 인프라 구축은 하나의 통합 스크립트(deploy-all-d4.sh)만 실행하면 완료된다.
다만, 동일한 작업 흐름이 01~05의 개별 스크립트로도 분리되어 있어 필요한 경우 아래와 같은 스크립트를 선택 실행할 수 있다.

1.2. 필수 도구 설치

```
$ ./deploy/01-install-tools.sh
```

2. 배포 환경 활용
2.1. 클러스터 컨텍스트 확인

```
$ kubectl config get-contexts
```

2.2. 기본 리소스 확인

```
$ kubectl get ns
$ kubectl get nodes
$ kubectl get pods -A
$ aws eks list-access-entries --cluster-name <CLUSTER_NAME>
```

3. D4 인프라 제거(옵션)
3.1 전체 자동 삭제

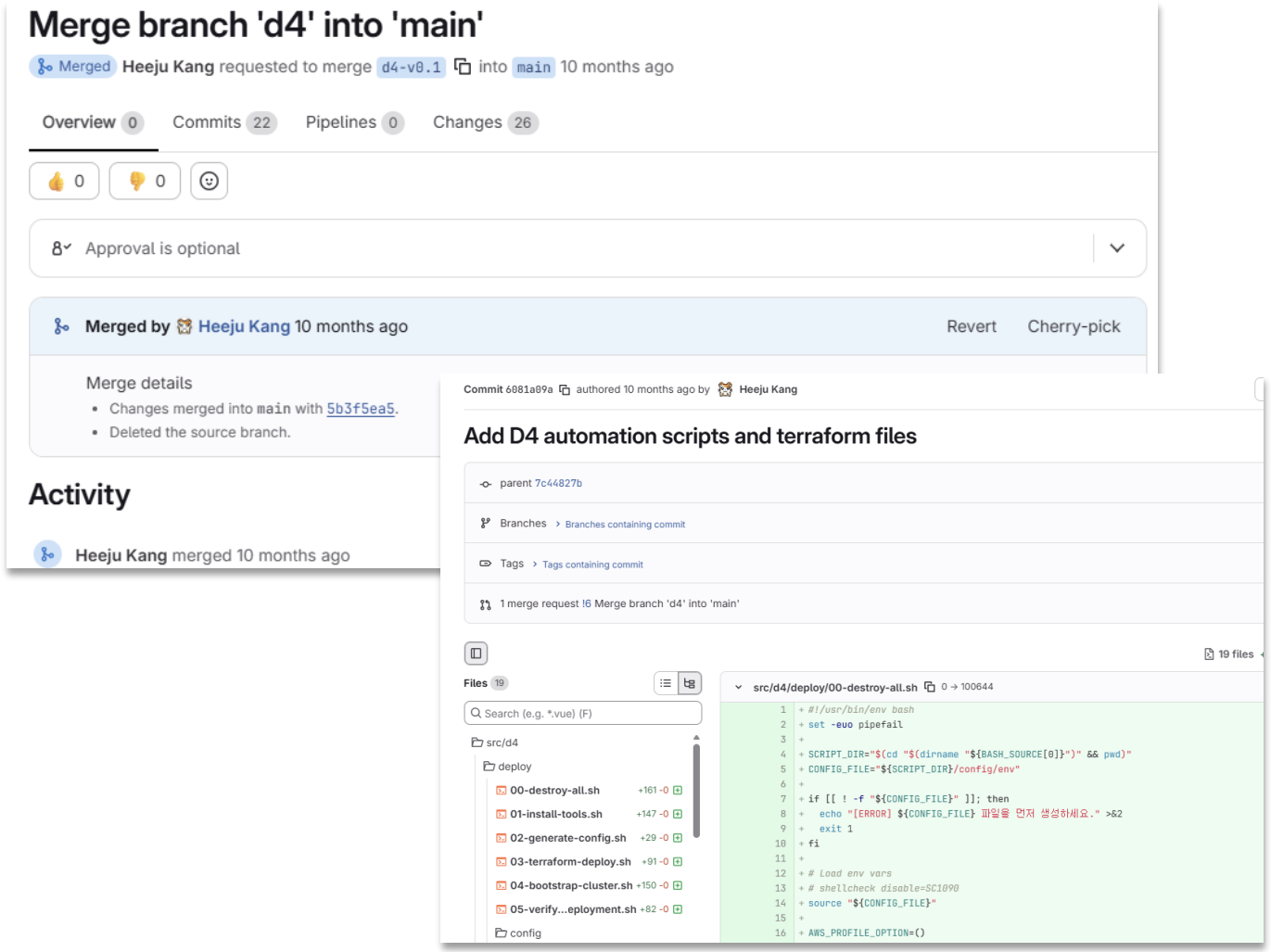
```
$ ./00-destroy-all.sh
```

제거하는 항목
- EKS 클러스터 / VPC / Subnet / Route Table / NAT Gateway / EBS CSI 드라이버 / 보안 그룹
- IAM AccessEntry

02 Terraform 기반 AWS EKS 인프라 구축 및 운영 자동화

GitLab 기반 협업

별도 Branch에서 인프라 코드 · 스크립트 관리, Merge Request를 통해 통합



배포 오류 분석 및 지원

증상
EKS 이관 후 Pod ImagePullBackOff

확인
kubectl describe Event에서 ECR image not found 확인

조치 및 결과
ECR Repository 및 해당 담당자 ECR 접근 권한 구성
이후 이미지 push 및 YAML 반영 후 정상 배포 확인

```
heeju@DESKTOP-AMPRB3A:~/ETRI-2025/src/d1/tenant-operator$ kubectl get pod -n d
NAME                                        READY   STATUS             RESTAR
tenant-admin-web-6b8846d56b-5bl48           0/1     ImagePullBackOff   0
tenant-operator-controller-5989bfd4c-nbmqd  0/1     ImagePullBackOff   0
```

```
heeju@DESKTOP-AMPRB3A:~/ETRI-2025/src/d1$ kubectl get pods -n d1-de
NAME                                        READY   STATUS    RESTA
tenant-admin-web-5c9677d658-7mknk           1/1     Running   0
tenant-operator-controller-7b96d7dc-b2jd4   1/1     Running   0

heeju@DESKTOP-AMPRB3A:~/ETRI-2025/src/d1$ kubectl get nodes -n d1-d
NAME                  STATUS   ROLES    AGE     VERSION
i-05bbc637da7320dfc   Ready    <none>   3h44m   v1.33.4-eks-e386d34
```

직무 역량 요약

01

Kubernetes & Container Platform

CRD · Operator · StatefulSet · Service · PV/PVC 기반 플랫폼 설계 및 운영

02

Automation

Terraform · Shell Script 기반 운영 자동화

03

Troubleshooting

로그 · Event · 리소스 상태 기반 문제 원인 단계적 추적

04

Cloud & Stateful Storage

AWS EKS · EBS CSI · Longhorn 기반 환경 구성

05

Collaboration & Documentation

GitLab 기반 협업 · 서비스 배포 지원 · 설치 및 운영 절차 문서화

Working Style

증상에서 멈추지 않고 로그 · 리소스 상태 · 실행 시점을 나눠 원인을 좁히고, 해결 과정은 문서로 기록해 재발 대응력을 높입니다.

처음 접하는 기술도 공식 문서와 실제 환경에서 검증하며 동작 구조를 이해합니다.

자격 사항

관련 기술의 기본 원리와 운영 역량을 자격 취득을 통해 검증했습니다



CKA
Kubernetes Administration



RHCE
Linux Automation & Administration



RHCSA
Linux System Administration